

Lecture 07 - Visibility Determination

This lecture asks which surfaces are visible from a viewpoint. Rasterization can generate many fragment candidates, but visibility decides which ones should affect the image.

Source chunk	course_text_parts/03_lectures/07_visibility-determination.txt
Extracted slide pages	69
Reader purpose	Commentary and causal explanation for human reading

How to read this lecture

- Read the section commentary before looking at the raw slide bullets.
- Use the mental model as the short version you should remember.
- Answer the check question without notes before continuing.

Slide walkthrough

Each row gives a short reading comment for one extracted slide page. Use it to turn sparse slide bullets into a human-readable path through the lecture.

Page	Slide title	How to read it
1	Untitled slide	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
2	Visibility Determination	Read this as camera geometry. Track how view-space positions become clip, normalized, and screen coordinates.
3	Object-Based Algorithms	Read this as camera geometry. Track how view-space positions become clip, normalized, and screen coordinates.
4	Image-Based Algorithms	Read this as continuous-to-discrete conversion. Ask which samples are covered and which attributes are interpolated.
5	7.1 Object-Based Algorithms	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
6	7.1 Object-Based Algorithms	Read this as boundary logic. Identify what is inside, what is outside, and when new intersection vertices are created.
7	Painter's Algorithm 1/3	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
8	Painter's Algorithm 2/3	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?

Page	Slide title	How to read it
9	Painter's Algorithm 3/3	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
10	Weiler-Atherton Algorithm 1/3	Read this as boundary logic. Identify what is inside, what is outside, and when new intersection vertices are created.
11	Weiler-Atherton Algorithm 2/3	Read this as boundary logic. Identify what is inside, what is outside, and when new intersection vertices are created.
12	Weiler-Atherton Algorithm 3/3	Read this as boundary logic. Identify what is inside, what is outside, and when new intersection vertices are created.
13	Back Face Culling 1/2	Read this as camera geometry. Track how view-space positions become clip, normalized, and screen coordinates.
14	Back Face Culling 2/2	Read this as the main data-flow story: scene/model data is processed step by step until only valid framebuffer updates remain.
15	Robert's Algorithm	Read this as continuous-to-discrete conversion. Ask which samples are covered and which attributes are interpolated.
16	Assessment Robert's Algorithm	Read this as continuous-to-discrete conversion. Ask which samples are covered and which attributes are interpolated.
17	7.2 Binary Space Partitioning	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
18	Spatial Data Structures	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
19	BSP Trees 1/2	Read this as an occlusion decision. Decide whether the method works in object space, image space, or per-fragment depth space.
20	BSP Trees 2/2	Read this as an occlusion decision. Decide whether the method works in object space, image space, or per-fragment depth space.
21	Node Data Structure	Read this as an occlusion decision. Decide whether the method works in object space, image space, or per-fragment depth space.
22	BSP Tree Construction	Read this as an occlusion decision. Decide whether the method works in object space, image space, or per-fragment depth space.
23	BSP Tree Construction - Example	Read this as an occlusion decision. Decide whether the method works in object space, image space, or per-fragment depth space.

Page	Slide title	How to read it
24	BSP Tree Construction – Pseudo Code	Read this as an occlusion decision. Decide whether the method works in object space, image space, or per-fragment depth space.
25	Visibility Determination	Read this as camera geometry. Track how view-space positions become clip, normalized, and screen coordinates.
26	Visibility Determination – Pseudo Code	Read this as an occlusion decision. Decide whether the method works in object space, image space, or per-fragment depth space.
27	BSP Tree Traversal - Example	Read this as an occlusion decision. Decide whether the method works in object space, image space, or per-fragment depth space.
28	Splitting Planes Selection	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
29	Traversal Optimization	Read this as an occlusion decision. Decide whether the method works in object space, image space, or per-fragment depth space.
30	Assessment BSP Trees	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
31	7.3 Warnock Algorithm	Read this as an occlusion decision. Decide whether the method works in object space, image space, or per-fragment depth space.
32	Warnock Algorithm	Read this as camera geometry. Track how view-space positions become clip, normalized, and screen coordinates.
33	Region-Based Visibility Determination	Read this as camera geometry. Track how view-space positions become clip, normalized, and screen coordinates.
34	Procedure	Read this as camera geometry. Track how view-space positions become clip, normalized, and screen coordinates.
35	Pseudo Code	Read this as camera geometry. Track how view-space positions become clip, normalized, and screen coordinates.
36	Assessment Warnock Algorithm	Read this as an occlusion decision. Decide whether the method works in object space, image space, or per-fragment depth space.
37	7.4 Depth Buffer Algorithm	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
38	Depth Buffer Algorithm 1/3	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.

Page	Slide title	How to read it
39	Depth Buffer Algorithm 2/3	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
40	Depth Buffer Algorithm 3/3	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
41	Pseudo Code – Depth Buffer Test	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
42	OpenGL Depth Buffer	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
43	Depth Buffer Precision	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
44	Z-Fighting 1/2	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
45	Z-Fighting 2/2	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
46	Assessment Depth Buffering	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
47	7.5 Depth Buffer Extensions	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
48	Semi-Transparent Objects 1/2	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
49	Semi-Transparent Objects 2/2	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
50	Order-Independent Transparency 1/4	Read this as an occlusion decision. Decide whether the method works in object space, image space, or per-fragment depth space.
51	Order-Independent Transparency 2/4	Read this as an occlusion decision. Decide whether the method works in object space, image space, or per-fragment depth space.
52	Order-Independent Transparency 3/4	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
53	Order-Independent Transparency 4/4	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?

Page	Slide title	How to read it
54	Generating Haloes 1/2	Read this as local shading. Name the normal, light vector, view vector, material term, and where the computation happens.
55	Generating Haloes 2/2	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
56	7.6 Ray Casting	Read this as an occlusion decision. Decide whether the method works in object space, image space, or per-fragment depth space.
57	Ray Casting Principle	Read this as camera geometry. Track how view-space positions become clip, normalized, and screen coordinates.
58	Ray Initialization 1/3	Read this as continuous-to-discrete conversion. Ask which samples are covered and which attributes are interpolated.
59	Ray Initialization 2/3	Read this as camera geometry. Track how view-space positions become clip, normalized, and screen coordinates.
60	Ray Initialization 3/3	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
61	Intersection Calculation	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
62	Implicit Surface Intersection Calculation	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
63	Intersection Calculation - Triangle	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
64	Pseudo Code – Analytic Intersection Computation	Read this as camera geometry. Track how view-space positions become clip, normalized, and screen coordinates.
65	Pseudo Code - Iterative Intersection Calculation (Ray Marching)	Read this as an occlusion decision. Decide whether the method works in object space, image space, or per-fragment depth space.
66	Assessment Ray Casting	Read this as an occlusion decision. Decide whether the method works in object space, image space, or per-fragment depth space.
67	but are crucial for accurate visibility determination	Read this as camera geometry. Track how view-space positions become clip, normalized, and screen coordinates.
68	Literature and other sources used in this chapter	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?

Page	Slide title	How to read it
69	Practice, Addison-Wesley.	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?

07.1 Object-Based Algorithms

Source location: Section 7.1

Commentary

Object-based visibility algorithms reason about geometry before or while comparing surfaces. They can sort, split, or reject objects based on spatial relations. This is different from simply letting every fragment fight in the depth buffer.

The value of object-based reasoning is reducing work or establishing correct order. The difficulty is that geometry can overlap in complicated ways, so simple global ordering is not always possible.

Mental model

Object-based visibility tries to solve visibility with geometry before final pixels are written.

Check yourself

Why can intersecting polygons make simple depth sorting fail?

07.2 Binary Space Partitioning

Source location: Section 7.2

Commentary

A BSP tree recursively divides space with planes. Once built, it can help traverse geometry in a view-dependent order. This is useful for visibility and ordering because the tree encodes spatial relationships.

The cost is preprocessing and possible splitting of geometry. BSP is a good example of trading memory and setup work for faster or more structured visibility decisions later.

Mental model

A BSP tree stores a recursive answer to 'which side of this plane is the geometry on?'

Check yourself

Why can building a BSP tree require splitting polygons?

07.3 Warnock Algorithm

Source location: Section 7.3

Commentary

Warnock's algorithm works in image space by subdividing regions until visibility is simple enough to decide. If a region is ambiguous, split it. If it is simple, fill it.

This illustrates a general graphics strategy: recursively reduce a hard global problem into smaller local problems. The algorithm is less central in modern OpenGL practice than the depth buffer, but it helps contrast image-space and object-space approaches.

Mental model

When a region is too complicated, subdivide until the answer becomes simple.

Check yourself

What makes Warnock's algorithm image-space rather than object-space?

07.4 Depth Buffer Algorithm

Source location: Section 7.4

Commentary

The depth buffer stores the closest accepted depth at each sample or pixel. For each fragment, compare its depth to the stored depth. If it passes, update the color and depth; otherwise discard it.

The strength of the depth buffer is simplicity and hardware efficiency. The weakness is precision: depth values are finite, and projection distributes precision unevenly. Bad near/far settings can create z-fighting.

Mental model

The depth buffer is a per-sample competition for closest visible fragment.

Check yourself

Why should the near plane not be unnecessarily close to the camera?

07.5 Depth Extensions and Ray Casting

Source location: Sections 7.5-7.6

Commentary

Depth buffer extensions refine or adapt depth-based visibility, for example by handling precision, transparency, or multiple layers more carefully. Standard depth testing alone is not a complete solution for every visibility situation.

Ray casting takes the opposite direction from rasterization: for each image sample, cast a ray into the scene and find the closest intersection. This naturally solves primary visibility but requires intersection work instead of raster coverage work.

Mental model

Rasterization pushes primitives to pixels; ray casting pulls visibility from pixels into the scene.

Check yourself

Why is transparency harder than opaque nearest-surface visibility?

End-of-lecture summary

If you remember only one thing from Lecture 07, remember this: This lecture asks which surfaces are visible from a viewpoint. Rasterization can generate many fragment candidates, but visibility decides which ones should affect the image.

Before moving on, you should be able to explain the lecture without reading slide bullets aloud. Use the vocabulary from the slides, but add causal links: why a step exists, what data it consumes, what it produces, and what can go wrong.